

10. [문제풀이] 오래된 Manifest 파일 수정하기

- 문제

TASK

A team tries to apply the **hpa** manifest, which was originally created on an older cluster located in `~/old/hpa.yaml`

Identify the correct API version that should be used.

Update the manifest to work on Kubernetes v1.35.
Verify that the **HPA** is created successfully.

TASK

A team tries to apply the **deployment** manifest, which was originally created on an older cluster located in `~/old/deploy.yaml`

Identify the correct API version that should be used.

Update the manifest to work on Kubernetes v1.35.
Verify that the **Deployment** is created successfully.

- 이전에 만들어놓은 아래 manifest 파일들을, k8s v1.35 버전에서 정상 동작하도록 수정
 - HPA의 yaml파일
 - Deployment의 yaml파일
- 정답 -
 - HPA의 yaml 파일을 아래와 같이 수정

```
apiVersion: autoscaling/v2 # autoscaling/v2beta1에서 수정
kind: HorizontalPodAutoscaler
metadata:
  name: web-hpa
spec:
```

```

scaleTargetRef:
  apiVersion: apps/v1
  kind: Deployment
  name: web-app
minReplicas: 1
maxReplicas: 5
metrics:
  - type: Resource
    resource:
      name: cpu
      # targetAverageUtilization: 70 에서 수정
      target:
        type: Utilization
        averageUtilization: 70

```

- 'hpa' 라고 검색하면 나오는 페이지 중, 3번째 문서에 yaml 예시가 있음. 해당 문서 참조하여, 수정
 - <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/>

🔍 Search this site

Search Results

About 501 results (0.12 seconds)

Horizontal Pod Autoscaling - Kubernetes

Kubernetes > workloads > autoscaling > horizontal-pod-autoscale

Mar 15, 2026 ... In Kubernetes, a HorizontalPodAutoscaler automatically updates a workload resource (such as a Deployment or StatefulSet), with the aim of ...

Horizontal Pod Autoscaling - Kubernetes

Kubernetes > docs > tasks > run-application > horizontal-pod-autoscale

Jan 7, 2026 ... 워크로드 스케일링의 안정성. HorizontalPodAutoscaler를 사용하여 레플리카 그룹의 크기를 관리할 때, 측정하는 메트릭의 동적 특성 때문에 레플리카 ...

HorizontalPodAutoscaler Walkthrough - Kubernetes

Kubernetes > run-application > horizontal-pod-autoscale-walkthrough

Mar 15, 2026 ... A HorizontalPodAutoscaler (**HPA** for short) automatically updates a workload resource (such as a Deployment or StatefulSet), with the aim of ...

- 수정사항

1. apiVersion을 'autoscaling/v2beta1' → 'autoscaling/v2'로 수정

apiVersion: Group/Version

Core Group: (v1)

- Pod, Service, ConfigMap, Secret, Namespace, Node, PersistentVolume

Named Group: (apps/v1)

- **apps**: Deployment, StatefulSet
- **autoscaling**: HPA
- **batch**: Job, CronJob
- **networking.k8s.io**: Ingress, NetworkPolicy
- **storage.k8s.io**: StorageClass

Alpha(v1alpha1): 초기 단계, 테스트 용도 (기본적으로 비활성화)

Beta(v1beta1): 사용 가능, 세부 사항은 바뀔 수 있지만 큰 틀은 유지됨

Stable(v1): 안정화 버전

API

Pod: /api/v1/namespaces/{ns}/pods

Service: /api/v1/namespaces/{ns}/services

Deployment: /apis/apps/v1/namespaces/{ns}/deployments

HPA: /apis/autoscaling/v2/namespaces/{ns}/horizontalpodautoscalers

- HPA는 'autoscaling' group에 속함.
- 기존 yaml의 버전인 'v2beta1'에서, 'v2'로 버전업 되었다.

2. targetAverageUtilization 작성 방법 변경

- HPA: 트래픽에 따라 pod 개수를 자동으로 늘이거나 줄이기 위한 object
- averageUtilization: 목표 CPU 사용률
 - 아래와 같이 지정하면, CPU 사용률이 70%로 유지되도록, HPA가 pod수를 관리한다.

```
target:
  type: Utilization
  averageUtilization: 70
```

- 정답 - Deployment
 - Deployment의 yaml 파일을 아래와 같이 수정

```
apiVersion: apps/v1 # extensions/v1beta1 에서 수정
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 1
  selector: # selector 필드가 없던 것을 수정
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: app
          image: lpro/exam
          ports:
            - containerPort: 8080
      strategy:
        type: RollingUpdate
```

```
rollingUpdate:
  maxSurge: "25%" # "invalid" 로 되어있던 것을, 기본값 "25%"입력
  maxUnavailable: "25%" # "invalid" 로 되어있던 것을, 기본값 "25%"입력
```

- Deploy 문서의 예시 yaml파일을 참조하여 수정
- 수정사항
 1. apiVersion을 'extensions/v1beta1' → 'apps/v1' 로 수정
 - group과 버전명이 모두 바뀜
 2. selector 필드 지정
 - 예전에는 selector 필드를 비워놓으면 template.metadata.labels 에서 자동으로 유추해 줬는데 필수로 변경되었다고 함
 3. maxSurge, maxUnavailable 값을 기본값으로 "25%"로 입력
 - deploy의 문서에서 기본값을 찾아서 입력
 - 또는, 아래와 같이 kubectl explain 명령어로, 필드 설명과 기본값을 확인 가능

```
[candidate@k8s-master old]$ kubectl explain deployment.spec.strategy.rollingUpdate
GROUP:      apps
KIND:       Deployment
VERSION:    v1

FIELD: rollingUpdate <RollingUpdateDeployment>
```

```
DESCRIPTION:
  Rolling update config params. Present only if DeploymentStrategyType =
  RollingUpdate.
  Spec to control the desired behavior of rolling update.
```

```
FIELDS:
  maxSurge      <IntOrString>
    The maximum number of pods that can be scheduled above the desired number of
    pods. Value can be an absolute number (ex: 5) or a percentage of desired
    pods (ex: 10%). This can not be 0 if MaxUnavailable is 0. Absolute number is
    calculated from percentage by rounding up. Defaults to 25%. Example: when
    this is set to 30%, the new ReplicaSet can be scaled up immediately when the
    rolling update starts, such that the total number of old and new
```

pods do not exceed 130% of desired pods. Once old pods have been killed, new ReplicaSet can be scaled up further, ensuring that total number of pods running at any time during the update is at most 130% of desired pods.

maxUnavailable <IntOrString>
 The maximum number of pods that can be unavailable during the update. Value can be an absolute number (ex: 5) or a percentage of desired pods (ex: 10%). Absolute number is calculated from percentage by rounding down. This can not be 0 if MaxSurge is 0. Defaults to 25%. Example: when this is set to 30%, the old ReplicaSet can be scaled down to 70% of desired pods immediately when the rolling update starts. Once new pods are ready, old ReplicaSet can be scaled down further, followed by scaling up the new ReplicaSet, ensuring that the total number of pods available at all times during the update is at least 70% of desired pods.